Universidad Tecnológica de León

Ser, saber, hacer

Recursividad

PROLOFU

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

ESPECIALIDAD: CIENCIA DE DATOS

GRUPO: LICD401

ALUMNO: Angel Alberto Gonzalez Soto

DOCENTE: Cristian Jesús Estrada Andrade

Universidad Tecnológica de León

http://www.utleon.edu.mx

Blvd. Universidad Tecnológica # 225

Col. San Carlos. Cp. 37670, León, Gto. Mex.

Tel (477) 7 10 00 20. Fax (477) 7 72 53 40 ©2021

León, Guanajuato, México, 30 de septiembre del 2026

Recursividad de colas

```
¿Qué es la recursión en cola?
Recursión en cola (Tail Recursion):
Es un tipo de recursión donde la llamada a sí misma es la última acción
que realiza la función. No queda ninguna operación matemática
ni lógica pendiente al regresar; el resultado final se va construyendo
sobre la marcha y se pasa en un argumento acumulador.


Cuadro comparativo:

Característica                Recursión Normal (Sin cola)                          Recursión en Cola

Última instrucción            Una operación pendiente (ej. elemento + funcion(...))     Solo la llamada recursiva directa

¿Cuándo se calcula
el resultado?                 "Al regreso (de reversa, desempilando llamadas)"     En el camino de ida (dentro del acumulador)

Memoria (Pila de llamadas)    Ocupa espacio por cada llamada: O(n)                 Reutiliza el marco de memoria: O(1)

Uso de acumulador             No utiliza                                           "Sí, es la clave para pasar el resultado parcial"

"""
```

Ejemplos:

```python
#1. Sumar los elementos de una lista

#Versión Normal (deja una suma pendiente al regreso):
def sumar(lista):
    if not lista:
        return 0
    # Queda pendiente la suma con el primer elemento
    return lista[0] + sumar(lista[1:])

# Versión en Cola (el acumulador lleva el total acumulado):
def sumar_cola(lista, total=0):
    if not lista:
        return total
    # La suma se hace en el argumento antes de llamar
    return sumar_cola(lista[1:], total + lista[0])

"""
En la versión normal, el primer número espera a que todos los demás se
sumen.
En la versión en cola, sumas el número actual a tu "bolsa" (total)
y le pasas la bolsa al siguiente.
"""

# 2. Encontrar el número máximo en una lista

# Versión Normal (compara de regreso)
def encontrar_max(lista):
    if len(lista) == 1:
        return lista[0]

    # La máquina NO sabe el máximo del resto de la lista todavía.
    # Se congela aquí esperando la respuesta para recién llamar a max(...)
    max_del_resto = encontrar_max(lista[1:])
    return lista[0] if lista[0] > max_del_resto else max_del_resto

"""
El problema: Si se tiene una lista de 1000 números,
la máquina congela 999 llamadas en memoria antes de hacer una sola
comparación.
```

```
"""

# Versión en Cola (lleva el "campeón actual" en el acumulador)
def encontrar_max_cola(lista, actual_max=None):
    if actual_max is None:
        actual_max = lista[0]
        lista = lista[1:]

    if not lista:
        return actual_max

    # Decides el ganador en el momento y se lo pasas al siguiente
    nuevo_max = lista[0] if lista[0] > actual_max else actual_max
    return encontrar_max_cola(lista[1:], nuevo_max)

"""
Por qué sirve: Es exactamente como buscarías en la vida real:
miras el primer número, si el siguiente es más grande lo recuerdas
(nuevo_max),
y descartas el anterior sin retener nada en la memoria.
"""

# 3. Búsqueda Binaria (Binary Search)
"""
El algoritmo por excelencia para buscar un elemento en un arreglo ordenado
en tiempo O(\log n) en lugar de buscar uno por uno.

Versión en Cola Directa
La búsqueda binaria es el mejor ejemplo de que los algoritmos eficientes
en computación son naturalmente recursivos de cola:
"""
def busqueda_binaria_cola(lista, objetivo, inicio=0, fin=None):
    if fin is None:
        fin = len(lista) - 1

    if inicio > fin:
        return -1  # No encontrado

    medio = (inicio + fin) // 2

    if lista[medio] == objetivo:
        return medio  # Encontrado: retorna la posición
```

```
    elif lista[medio] > objetivo:
        # Busca en la mitad izquierda
        return busqueda_binaria_cola(lista, objetivo, inicio, medio - 1)
    else:
        # Busca en la mitad derecha
        return busqueda_binaria_cola(lista, objetivo, medio + 1, fin)
```